



С# расскажи о паттерне проектирования "Шаблонный метод"

Расскажи плюсы, минусы

Когда и где применяется

Утратил ли этот паттерн свою актуальность

Приведи примеры кода

Паттерн «Шаблонный метод» (Template Method) — это поведенческий ООП-паттерн, в котором базовый класс определяет «скелет» алгоритма, а подклассы переопределяют отдельные его шаги, не меняя общей структуры. В С# это обычно реализуется как абстрактный (или частично абстрактный) класс с финальным публичным методом, внутри которого вызываются виртуальные/абстрактные методы-шаги. ^[1] ^[2]

Что такое «Шаблонный метод»

Назначение:

- Определить шаблон (каркас) алгоритма в базовом классе.
- Выделить шаги, которые могут различаться в подклассах.
- Сделать эти шаги виртуальными или абстрактными, чтобы наследники могли их переопределять. ^[2] ^[1]

Классическая структура:

```
public abstract class Processor
{
    // Шаблонный метод — финальный, клиенты работают только с ним
    public void Process()
    {
        Prepare();
        var data = LoadData();
        Transform(data);
        SaveData(data);
        Cleanup();
    }

    // Стандартный шаг — можно переопределять, но не обязательно
    protected virtual void Prepare()
    {
        // базовая логика
    }

    // Обязательный шаг — переопределяется в подклассах
}
```

```

protected abstract long LoadData();

// optional шаг
protected virtual void Transform(long data)
{
    // базовая трансформация
}

// another обязательный шаг
protected abstract void SaveData(long data);

// cleanup
protected virtual void Cleanup()
{
    // базовая очистка
}
}

public class FileProcessor : Processor
{
    protected override long LoadData()
    {
        // читаем из файла
        return 123;
    }

    protected override void SaveData(long data)
    {
        // записываем в файл
    }

    protected override void Transform(long data)
    {
        // специфичная трансформация для файлов
    }
}

```

Клиент работает только с `Process()`, а детали реализации шагов скрыты в подклассах. ^[1]

^[2]

Плюсы паттерна

1. Устранение дублирования кода

Общий алгоритм вынесен в базовый класс, подклассы содержат только отличия. ^[1]

2. Единая точка контроля порядка шагов

Порядок выполнения шагов фиксируется в шаблонном методе, его нельзя случайно «перепутать» в подклассах. ^[2]

3. Простое расширение

Чтобы добавить новую вариацию алгоритма, нужно просто создать новый подкласс и переопределить нужные шаги. ^[1]

4. Инкапсуляция алгоритма

Клиенты не знают деталей реализации шагов, взаимодействуют только с шаблонным

методом.^[2]

Минусы паттерна

1. Нарушение принципа инверсии зависимостей (в классическом виде)

Подклассы зависят от базового класса, а не наоборот; изменение базового класса может ломать много наследников.^[2]

2. Сложность при большом числе шагов

Если алгоритм многошаговый и с множеством вариантов, абстрактный класс может стать громоздким и трудным для понимания.^[2]

3. Ограниченная гибкость по сравнению с «Стратегией»

Шаблонный метод работает через наследование, а не через передачу объектов-стратегий; изменить поведение в runtime сложнее.^[1]

4. Риск «тяжелых» базовых классов

При частом дроблении шагов и их вариаций базовый класс может превращаться в «монолит» с множеством виртуальных методов.^[2]

Когда и где применяется

Паттерн удобен, когда:

- Есть схожие алгоритмы с одинаковой общей структурой, но разными деталями отдельных шагов.^[1]
- Нужно фиксировать порядок шагов и гарантировать, что подклассы не будут его нарушать.^[2]
- В будущем планируется создавать много подклассов, каждый со своей вариацией алгоритма.^[1]
- В коде есть дублирование похожих алгоритмов в разных классах — их можно объединить в шаблонный метод.^[1]

Примеры использования в .NET:

- WCF: классы вроде `CommunicationObject`, `ChannelBase`, `ServiceHostBase` используют шаблонный метод: публичные методы `Open`, `Close` и т.п. `sealed`, но внутри вызывают виртуальные/абстрактные `OnOpen`, `OnClose`.^[1]
- Любой абстрактный класс с защищёнными абстрактными/виртуальными методами, которые вызываются из общего публичного метода, фактически является примером `Template Method`.^[1]

Утратил ли паттерн актуальность в C#

Нет, паттерн не утратил актуальность, но его применение стало более осознанным:

- В современных подходах (чистая архитектура, композиция вместо наследования) часто предпочитают стратегию + делегирование вместо глубоких базовых классов с множеством виртуальных шагов.^[3]

- Однако для фреймворков и базовых классов (например, сервисы обработки, workflow-механизмы) Template Method остаётся удобным способом предоставить «скелет» алгоритма и позволяя пользователям переопределять детали.^[3] ^[1]
- В .NET Aspire, Dapr-подобных архитектурах и библиотечных абстракциях этот паттерн всё ещё активно используется для построения гибких, но контролируемых алгоритмов.^[3]

Таким образом, паттерн остался важным инструментом, особенно в библиотеках и фреймворках, но в простых приложениях его часто заменяют более композиционными решениями.

Примеры кода в контексте .NET

1. Обработка данных (файл, блог, API)

```
public abstract class DataPipeline<T>
{
    public void Run()
    {
        var source = LoadSource();
        var items = Parse(source);
        items = Transform(items);
        Save(items);
    }

    protected abstract T LoadSource();
    protected abstract IEnumerable<string> Parse(T source);
    protected virtual IEnumerable<string> Transform(IEnumerable<string> items) => items;
    protected abstract void Save(IEnumerable<string> items);
}

public class FilePipeline : DataPipeline<string>
{
    protected override string LoadSource()
    {
        // чтение файла
        return System.IO.File.ReadAllText("input.txt");
    }

    protected override IEnumerable<string> Parse(string source)
    {
        return source.Split('\n');
    }

    protected override void Save(IEnumerable<string> items)
    {
        System.IO.File.WriteAllLines("output.txt", items);
    }
}
```

2.wurf-сервис (веб, консоль, тест)

```
public abstract class ReportService
{
    public void GenerateReport()
    {
        var data = CollectData();
        Format(data);
        Export(data);
    }

    protected abstract IEnumerable<DataItem> CollectData();
    protected virtual void Format(IEnumerable<DataItem> data) { }
    protected abstract void Export(IEnumerable<DataItem> data);
}

public class WebReportService : ReportService
{
    protected override IEnumerable<DataItem> CollectData()
    {
        // сбор данных из API / БД
        return new List<DataItem>();
    }

    protected override void Export(IEnumerable<DataItem> data)
    {
        // возврат HTML / JSON клиенту
    }
}
```

Эти примеры показывают типичное применение: один публичный метод, набор шагов, часть обязательных, часть переопределяемых по желанию.^{[2] [1]}

Если кратко:

- Шаблонный метод — это «каркас алгоритма + переопределяемые шаги».
- Плюсы: устранение дублирования, контроль порядка, простое расширение.
- Минусы: сильная связь через наследование, риск громоздких базовых классов, меньше гибкости чем у Стратегии.
- В современных .NET-приложениях он всё ещё актуален, особенно в фреймворках и библиотеках, но в простых сценариях часто заменяется композицией и делегированием.

✱✱

Чем отличается «Стратегия» от «Шаблонного метода»

Главное различие: как меняется поведение алгоритма.

- **Шаблонный метод (Template Method)**
 - Поведение меняется через наследование: подкласс переопределяет виртуальные/абстрактные шаги.
 - Структура алгоритма фиксирована в базовом классе, шаги могут быть переопределены, но порядок — никогда. ^[11] ^[12]
- **Стратегия (Strategy)**
 - Поведение меняется через делегирование: объект хранит ссылку на объект-стратегию и «передает» ему часть работы.
 - Алгоритм (или его существенная часть) полностью вынесен в отдельный класс, который можно менять динамически. ^[13] ^[14]

Табличное сравнение:

Аспект	Шаблонный метод	Стратегия
Способ изменения поведения	Наследование (virtual/abstract методы)	Делегирование (интерфейс/делиться + объект)
Гибкость в runtime	Низкая (нужен новый тип, компиляция)	Высокая (можно менять стратегию «на лету»)
Где логика алгоритма	В базовом классе + перегрузки в подклассах	В отдельных классах-стратегиях
Связность	Сильная связь «базовый ↔ подкласс»	Слабая связь «контекст ↔ интерфейс стратегии»
Когда удобно	Много похожих классов с небольшими отличиями	Много вариантов поведения, часто меняются

Из источников:

- Шаблонный метод использует наследование, чтобы расширять части алгоритма. Стратегия использует делегирование, чтобы изменять выполняемые алгоритмы. ^[11]
- В отличие от Шаблонного метода, Стратегия делегирует весь алгоритм отдельным классам. ^[13]

«Стратегия + делегирование» — что это

Это комбинация:

1. Стратегия:

- Семейство алгоритмов описывается через общий интерфейс (например IFormatter, IAuthStrategy, IExportStrategy).
- Каждый алгоритм — отдельный класс, реализующий этот интерфейс. ^[14] ^[13]

2. Делегирование:

- Клиент (контекст) не выполняет алгоритм сам, а передает его выполнение объекту-стратегии через метод, свойство или делегат (в C# это может быть `Func<>`, `Action<>`, `delegate` или интерфейс).^[15] ^[16]

Пример с интерфейсом:

```
public interface IReportStrategy
{
    string Generate(IEnumerable<DataItem> data);
}

public class CsvReportStrategy : IReportStrategy
{
    public string Generate(IEnumerable<DataItem> data)
    {
        // CSV-форматирование
    }
}

public class JsonReportStrategy : IReportStrategy
{
    public string Generate(IEnumerable<DataItem> data)
    {
        // JSON-форматирование
    }
}

public class ReportService
{
    private readonly IReportStrategy _strategy;

    public ReportService(IReportStrategy strategy)
    {
        _strategy = strategy; // делегирование выбора стратегии
    }

    public string CreateReport(IEnumerable<DataItem> data)
    {
        return _strategy.Generate(data);
    }
}
```

`ReportService` не знает, как именно формируется отчет — он просто делегирует это стратегии.^[14] ^[13]

Пример с делегатом (без явных классов-стратегий):

```
public class TextProcessor
{
    private readonly Func<string, string> _transform;

    public TextProcessor(Func<string, string> transform)
    {
        _transform = transform; // делегируем трансформацию
    }
}
```

```

    }

    public string Process(string input)
    {
        return _transform(input);
    }
}

// Использование:
var upper = new TextProcessor(s => s.ToUpper());
var lower = new TextProcessor(s => s.ToLower());

```

Тут вместо классов-стратегий используется делегат `Func<string, string>` как «легкая стратегия». [\[16\]](#) [\[15\]](#)

Чем это лучше «Шаблонного метода» и когда применяется

Преимущества стратегии + делегирования

1. Гибкость в runtime

- Стратегию можно менять динамически (например, через конфигурацию, DI, пользовательские настройки).
- В шаблонном методе для нового поведения нужен новый подкласс и, как правило, новая компиляция. [\[12\]](#) [\[13\]](#)

2. Композиция вместо наследования

- Контекст зависит от абстракции (`IStrategy`, делегат), а не от конкретного наследуемого типа.
- Это лучше соответствует принципам чистой архитектуры: зависимости инвертированы, тестирование проще [jasulib](#).

3. Избежание разветвленных условий

- Когда вместо:

```

if (type == "csv") ...
else if (type == "json") ...

```

есть набор стратегий и выбор одной из них. [\[13\]](#) [\[14\]](#)

4. Локализация сложности

- Мѳи алгоритм — отдельный класс/делегат, его можно тестировать и поддерживать отдельно.
- В шаблонном методе вся логика алгоритма часто «размазана» между базовым классом и множеством подклассов. [\[13\]](#)

5. Проще расширять без нарушения существующего кода

- Добавление новой стратегии не требует изменения базового класса или остальных подклассов (принцип открытости/закрытости). [\[13\]](#)

Где применяется в .NET-проектах

1. Бизнес-правила и вариативные алгоритмы

- Разные способы расчета скидок, начисления бонусов, проверки условий — каждая стратегия как отдельный класс. ^[14] ^[13]

2. Экспорт/импорт, форматы данных

- `IExportStrategy` для CSV, JSON, XML, PDF; сервис выбирает стратегию по конфигу или параметрам запроса. ^[13]

3. Аутентификация и авторизация

- `IAuthStrategy`: JWT, OAuth, базовая авторизация, локальный логин — контекст делегирует логин стратегию. ^[14]

4. Микросервисы и Dapr-подобные сценарии

- Разные стратегии `retrying`, блочной логики, маршрутизации, форматов сообщений — выбираются через конфигурацию или DI. ^[14]

5. Backend-сервисы с вариативной логикой

- Например, `BackgroundService`, который по конфигу может использовать одну из стратегий обработки событий (file, db, api). ^[14]

Краткое правило выбора

- Если:
 - алгоритм статичный, его структура не меняется,
 - варианты поведения — это разные подклассы с небольшими отличиями шагов,
 - не нужно менять стратегию в runtime,→ Шаблонный метод часто удобнее.
- Если:
 - нужно менять поведение динамически (конфиг, DI, пользовательские настройки),
 - есть много вариантов алгоритма, часто добавляются новые,
 - хотите избегать глубоких иерархий наследования и «тяжелых» базовых классов,→ Стратегия + делегирование предпочтительнее, особенно в чистых архитектурах и современных .NET-проектах [jasulib](#). ^[13] ^[14]

✱

Приветствуется ли применять в паттерне Стратегия несколько разных стратегий.

Например BackgroundService<TBehavior, TScheduler, TMetrics, TTuner>

TBehavior - поведение

TScheduler - временное расписание этого поведения

TMetrics - понятно

TTuner - мелкие параметры, лучшие настройки

?

Да, в паттерне «Стратегия» приветствуется использовать несколько разных стратегий одновременно, но с важным уточнением:

- Классический вид стратегии — одна стратегия на одно поведение (один алгоритм, один интерфейс).
- Когда у вас есть несколько независимых аспектов поведения, правильнее рассматривать это как композицию нескольких стратегий, а не одну «гипер-стратегию».

То есть ваш пример:

```
BackgroundService<TBehavior, TScheduler, TMetrics, TTuner>
```

можно реализовать как контекст с четырьмя разными стратегиями:

```
public class BackgroundService<TBehavior, TScheduler, TMetrics, TTuner>
    : BackgroundService
    where TBehavior : IBehaviorStrategy
    where TScheduler : ISchedulerStrategy
    where TMetrics : IMetricsStrategy
    where TTuner : ITunerStrategy
{
    private readonly TBehavior _behavior;
    private readonly TScheduler _scheduler;
    private readonly TMetrics _metrics;
    private readonly TTuner _tuner;

    public BackgroundService(
        TBehavior behavior,
        TScheduler scheduler,
        TMetrics metrics,
        TTuner tuner)
    {
        _behavior = behavior;
        _scheduler = scheduler;
        _metrics = metrics;
        _tuner = tuner;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
```

```

        // примеры делегирования:
        await _scheduler.ScheduleLoopAsync(stoppingToken, DoWork);
    }

    private async Task DoWork(Cancellation_token token)
    {
        _metrics.Begin();
        await _behavior.ExecuteAsync(token);
        _metrics.End();
        _tuner.ApplyAdjustments();
    }
}

```

Здесь:

- `IBehaviorStrategy` — что именно делает сервис (логика работы).
- `ISchedulerStrategy` — как и когда запускать (расписание, интервалы).
- `IMetricsStrategy` — как собирать метрики.
- `ITunerStrategy` — как применять параметры/настройки.

Это **не нарушение** стратегии, а классическое применение «стратегия для каждого аспекта». [\[21\]](#) [\[22\]](#) [\[23\]](#)

Чем это лучше одной «комплексной» стратегии

Если сделать одну стратегию, например:

```

public interface IBackgroundStrategy
{
    void DoBehavior();
    TimeSpan ScheduleInterval();
    void RecordMetric(string name, double value);
    void ApplyTuning();
}

```

то:

- Все аспекты запихнуты в один интерфейс — **нарушается модульность**.
- Новые варианты поведения требуют менять **весь класс стратегии**, даже если меняется только, например, метрики.
- Тестировать и комбинировать варианты сложнее: чтобы поменять только расписание, нужно переписать весь класс стратегии.

При `compositional` подходе (4 стратегии):

- Каждый аспект — отдельный интерфейс и набор классов.
- Можно легко комбинировать: `BehaviorA + SchedulerB + MetricsC + TunerD`.
- Расширение без изменения старого кода (принцип открытости/закрытости). [\[22\]](#) [\[21\]](#)

Когда это особенно удобно

Приветствуется, когда:

1. Аспекты поведения независимы

- Schedule, Behavior, Metrics, Tuning — разные домены, разные правила изменения.

2. Нужна гибкость конфигурации

- Через DI можно подбирать разные стратегии из конфига:
 - ScheduledEveryMinuteBehavior + DbScheduler + PrometheusMetrics + ConfigTuner.

3. Разные команды работают над разными аспектами

- Один мейнетит behavior, другой — scheduler, третий — metrics.

4. Требуется «вложенные» стратегии

- SchedulerStrategy может сам использовать ConditionStrategy или RetryStrategy.

Такой подход часто встречается в современных .NET-проектах, особенно в чистых архитектурах и микросервисах, где композиция и делегирование важны [jasulib](#).

Как это выглядит в коде с делегатами

Если не нужны отдельные классы для маленьких стратегий, можно тоже использовать делегаты:

```
public class BackgroundService
{
    private readonly Func<CancellationToken, Task> _behavior;
    private readonly Func(TimeSpan, CancellationToken, Task> _scheduler;
    private readonly Action<string, double> _metrics;
    private readonly Func<TunerConfig, TunerConfig> _tuner;

    public BackgroundService(
        Func<CancellationToken, Task> behavior,
        Func(TimeSpan, CancellationToken, Task> scheduler,
        Action<string, double> metrics,
        Func<TunerConfig, TunerConfig> tuner)
    {
        _behavior = behavior;
        _scheduler = scheduler;
        _metrics = metrics;
        _tuner = tuner;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        _scheduler(TimeSpan.FromSeconds(10), stoppingToken, async token =>
        {
            _metrics("start", 1);
            await _behavior(token);
            _metrics("end", 1);
        });
    }
}
```

```
}  
}
```

Это тоже «стратегия + делегирование», но с несколькими стратегиями (behavior, scheduler, metrics, tuner).^[24] ^[25]

Резюме

- Применять в контексте **несколько разных стратегий** — это хорошая практика, если они отвечают за независимые аспекты поведения.
- Ваш `BackgroundService<TBehavior, TScheduler, TMetrics, TTuner>` — это **не одна сложная стратегия**, а **четыре компактные стратегии**, каждая для своего аспекта.
- Такой подход:
 - Улучшает модульность и тестируемость.
 - Упрощает конфигурацию и комбинацию вариантов.
 - Лучше соответствует принципам чистой архитектуры и современным .NET-практикам [jasulib](#).^[21] ^[22]



Я не понял вот этот Ваш вариант правильный или нет ?

```
public class BackgroundService<TBehavior, TScheduler, TMetrics, TTuner> :  
    BackgroundService where TBehavior : IBehaviorStrategy where TScheduler :  
    ISchedulerStrategy where TMetrics : IMetricsStrategy where TTuner : ITunerStrategy { private  
    readonly TBehavior _behavior; private readonly TScheduler _scheduler; private readonly  
    TMetrics _metrics; private readonly TTuner _tuner; public BackgroundService( TBehavior  
    behavior, TScheduler scheduler, TMetrics metrics, TTuner tuner) { _behavior = behavior;  
    _scheduler = scheduler; _metrics = metrics; _tuner = tuner; }
```

Да, этот вариант **правильный** по своей идее и синтаксически корректный в C#. Просто он сокращен — в нём нет перезакрытия `ExecuteAsync`, но структура класса и типизация стратегий верны.

Что здесь правильно

```
public class BackgroundService<TBehavior, TScheduler, TMetrics, TTuner> : BackgroundS  
    where TBehavior : IBehaviorStrategy  
    where TScheduler : ISchedulerStrategy  
    where TMetrics : IMetricsStrategy  
    where TTuner : ITunerStrategy  
{  
    private readonly TBehavior _behavior;  
    private readonly TScheduler _scheduler;  
    private readonly TMetrics _metrics;  
    private readonly TTuner _tuner;
```

```

    public BackgroundService(
        TBehavior behavior,
        TScheduler scheduler,
        TMetrics metrics,
        TTuner tuner)
    {
        _behavior = behavior;
        _scheduler = scheduler;
        _metrics = metrics;
        _tuner = tuner;
    }

    // тут можно добавить ExecuteAsync
}

```

- Класс наследуется от BackgroundService ✓
- Использует 4 разных типа стратегий ✓
- Каждый тип ограничен соответствующим интерфейсом через where ✓
- Конструктор принимает стратегии и сохраняет их в поля ✓
- Синтаксис where T : IInterface в C# работает именно так ✓

Что нужно добавить, чтобы это стало рабочим сервисом

BackgroundService требует реализовать абстрактный метод ExecuteAsync. Например:

```

public class BackgroundService<TBehavior, TScheduler, TMetrics, TTuner> : BackgroundService
{
    where TBehavior : IBehaviorStrategy
    where TScheduler : ISchedulerStrategy
    where TMetrics : IMetricsStrategy
    where TTuner : ITunerStrategy

    private readonly TBehavior _behavior;
    private readonly TScheduler _scheduler;
    private readonly TMetrics _metrics;
    private readonly TTuner _tuner;

    public BackgroundService(
        TBehavior behavior,
        TScheduler scheduler,
        TMetrics metrics,
        TTuner tuner)
    {
        _behavior = behavior;
        _scheduler = scheduler;
        _metrics = metrics;
        _tuner = tuner;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        // пример: scheduler запускает цикл, внутри – behavior
    }
}

```

```

        await _scheduler.StartLoopAsync(stoppingToken, async token =>
        {
            _metrics.Begin();
            try
            {
                await _behavior.ExecuteAsync(token);
            }
            finally
            {
                _metrics.End();
            }

            _tuner.ApplyAdjustments();
        });
    }
}

```

Итог

- Ваш код **правильный** по структуре и типизации.
- Это **не сломанный пример**, а просто «заглушка» сервиса без ExecuteAsync.
- Добавив ExecuteAsync, вы получите полноценный фоновый сервис с 4 независимыми стратегиями (behavior, scheduler, metrics, tuner).

✱

А если этих SubStrategy будет 10 или 20, то это тоже будет нормально ?

Если у меня есть сложный объект, который рассматривается с разных сторон, то мы все равно не должны смешивать наши отдельные стратегии.
?

Да, даже при 10–20 sub-стратегиях это тоже **нормально** — если они разделены по доменам ответственности.

Но есть важный нюанс:

не смешивать все эти стратегии в один «монолитный» интерфейс, а **оставить их независимыми**.

Много стратегий — это нормально, если они независимы

Пример: ваш сложный объект рассматривается с разных сторон:

- Поведение (behavior)
- Расписание (scheduler)
- Метрики (metrics)
- Тунинг (tuner)

- Авторизация (auth)
- Логирование (logging)
- Кэширование (cache)
- Retry-политика (retry)
- Сериализация (serializer)
- Сжатие (compressor)
- и т.д.

Если у вас будет:

```
public class BackgroundService<
    TBehavior, TScheduler, TMetrics, TTuner,
    TAuth, TLogging, TCache, TRetry, TSer, TCompress>
    : BackgroundService
    where TBehavior : IBehaviorStrategy
    where TScheduler : ISchedulerStrategy
    where TMetrics : IMetricsStrategy
    where TTuner : ITunerStrategy
    where TAuth : IAuthStrategy
    where TLogging : ILoggingStrategy
    where TCache : ICacheStrategy
    where TRetry : IRetryStrategy
    where TSer : ISerializerStrategy
    where TCompress : ICompressorStrategy
{ ... }
```

он станет неудобным:

- Много типов параметров.
- Конструктор с 10+ аргументами.
- Сложность чтения и поддержки.

Поэтому при 10–20 стратегиях лучше не делать один класс с 10+ дженериками, а:

1. Разбить на несколько **компонентов** (например, `ProcessingService`, `SchedulingService`, `LoggingService`).
2. Или использовать **композицию через интерфейс** — собирать сложный сервис из нескольких небольших сервисов, каждый со своей стратегией.

Но принцип «не смешивать стратегии» остается: каждый аспект — отдельный интерфейс и отдельный набор стратегий.

Не смешивать отдельные стратегии — это правильно

Вы спросили:

Если у меня есть сложный объект, который рассматривается с разных сторон, то мы все равно не должны смешивать наши отдельные стратегии.

Да, не должны смешивать, и это правильно.

Почему:

1. Разная ответственность

- Scheduler отвечает за расписание, а Bridge не должен знать, как оно работает.
- Metrics отвечает за сбор метрик, а Behavior не должен за них заботиться.

2. Разная динамика изменений

- Те, кто меняет логику поведения, не должны дергать код метрик.
- Те, кто меняет способ кэширования, не должны менять код scheduler.

3. Принцип единой ответственности (SRP)

Каждый интерфейс стратегии — одна ответственность.
Смешивание нарушает SRP и усложняет тестирование.

4. Чистая архитектура

В чистой архитектуре интерфейсы (включая стратегии) должны быть маленькими и специализированными, а зависимости направлены от инфраструктуры к бизнес-логике. ^[41]

Как практично организовать 10–20 стратегий

Вариант 1: Несколько небольших сервисов

Не один класс с 20 дженериками, а несколько:

```
public class ProcessingService<TBehavior, TAuth, TRetry>
    : IHostedService
    where TBehavior : IBehaviorStrategy
    where TAuth : IAuthStrategy
    where TRetry : IRetryStrategy
{ ... }

public class SchedulingService<TScheduler>
    : IHostedService
    where TScheduler : ISchedulerStrategy
{ ... }

public class LoggingService<TLogging>
    : IHostedService
    where TLogging : ILoggingStrategy
{ ... }

public class MetricsService<TMetrics>
    : IHostedService
    where TMetrics : IMetricsStrategy
{ ... }
```

Тогда:

- Каждый сервис имеет мало стратегий (2–4).
- Конструкторы небольшие.
- Логика разделена по доменам.

В DI вы регистрируете их независимо:

```
services.AddHostedService<ProcessingService<MyBehavior, JwtAuth, DefaultRetry>>();
services.AddHostedService<SchedulingService<IntervalScheduler>>();
services.AddHostedService<LoggingService<FileLogger>>();
```

Вариант 2: Композитный сервис с «под-сервисами»

Если нужен один «главный» сервис, пусть он собирает другие:

```
public class MasterBackgroundService : BackgroundService
{
    private readonly ProcessingService _processingService;
    private readonly SchedulingService _schedulingService;
    private readonly MetricsPoller _metricsPoller;

    public MasterBackgroundService(
        ProcessingService processingService,
        SchedulingService schedulingService,
        MetricsPoller metricsPoller)
    {
        _processingService = processingService;
        _schedulingService = schedulingService;
        _metricsPoller = metricsPoller;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        // запускаем под-сервисы
        await Task.WhenAll(
            _processingService.StartAsync(stoppingToken),
            _schedulingService.StartAsync(stoppingToken),
            _metricsPoller.StartAsync(stoppingToken));
    }
}
```

Здесь:

- Универсальный BackgroundService не перегружен 20 дженериками.
- Каждый под-сервис — отдельная стратегия своей области.

Вариант 3: Функциональные стратегии (делегаты)

Для маленьких стратегий (логирование, метрики, retry) можно использовать делегаты:

```
public class ProcessingService
{
    private readonly Func<Task> _behavior;
    private readonly Action<string> _log;
    private readonly Action<string, double> _metric;

    public ProcessingService(
        Func<Task> behavior,
        Action<string> log,
        Action<string, double> metric)
    {
        _behavior = behavior;
        _log = log;
        _metric = metric;
    }
}
```

Тогда:

- 10–20 стратегий не требуют 10–20 дженериков.
- Вы просто **подсовываете делегаты**.

Это тоже «стратегия + делегирование», просто более легковесная.

Ответ на ваш вопрос

- 10–20 стратегий — это нормально, если они **разделены по доменам и не смешаны в один интерфейс**.
- Сложный объект, рассматриваемый с разных сторон, **не должен иметь одну «моноклитную» стратегию**, а должен использовать **композицию нескольких стратегий**.
- Подход вида:

```
BackgroundService<TBehavior, TScheduler, TMetrics, TTuner>
```

— хороший паттерн, но при 10–20 стратегиях лучше:

- Разбить на несколько сервисов, или
- Использовать делегаты для маленьких стратегий, или
- Собрать их через композицию через под-сервисы.

Это сохраняет **модульность, тестируемость и читаемость** и соответствует принципам чистой архитектуры. ^[41]

По своему опыту паттерн Strategy - один из лучших.

Я делал как раз много разных стратегий в одном контейнере Strategies.

Потом я все параметры полученные от разных стратегий подавал на AI - модель и получал очень хорошие результаты. А если бы я смешивал их внутри контейнера - ничего хорошего не получилось.

Так и есть: ваш подход — это классическая композиция независимых стратегий — и он очень хорошо работает в том числе для AI-интеграций.

Что именно здесь важно:

1. Разделенные стратегии дают AI «чистые признаки»

- Каждая стратегия отвечает за одну сторону объекта (behavior, scheduler, metrics, tuner, logging и т.д.).
- Модель получает структурированные, семантически понятные данные:
 - «что делалось» (behavior),
 - «как и когда» (scheduler),
 - «какие метрики» (metrics),
 - «какие настройки» (tuner) и т.п.
- Это гораздо лучше для обучения и интерпретации, чем «смесь» всех данных в одной куче.

2. Смешивание усложняет и размывает контекст

- Если все параметры слить в один контейнер без разделения, модель не видит четкой структуры:
 - Какие данные относятся к расписанию?
 - Какие к поведению?
 - Какие к настройкам?
- Это снижает качество предсказаний и усложняет интерпретацию результатов.

3. Композиция стратегий — это и архитектурно, и ML-правильно

- Архитектурно:
 - Каждый компонент имеет одну ответственность.
 - Тестирование, замена и изменение стратегий становятся проще.
- ML-правильно:
 - Модель получает «признаки с метками» (behavior, scheduler, metrics, tuner и т.д.).
 - Это линейно улучшает результат (как вы и заметили).

4. Вы фактически применили принцип «разделять и властвовать»

- Разделили задачу на:
 - Сбор данных по разным аспектам (стратегии).
 - Их агрегацию в единый контекст для AI.
- Это очень близкий к «чистой архитектуре» подход:
 - Domain/Use Cases: сбор данных через стратегии.
 - Infrastructure: передача их в AI-модель.
- И в результате модель видит не «серь», а структурированную картину системы.

Если бы вы смешали все внутри одного контейнера, вы бы:

- Усложнили код.
- Усложнили тестирование.
- Ухудшили readability для модели (и для людей).

Ваш опыт — хороший пример того, что:

- Много независимых стратегий — это не «переусложнение», а правильный способ моделирования сложных объектов.
- Не смешивать их — ключевой принцип, который напрямую влияет на качество AI-результатов.

Если хотите, можно дальше обсудить, как формализовать эту структуру для передачи в AI (например, через DTO, JSON-схему или специфичный формат признаков), чтобы еще улучшить интерпретируемость и качество модели.

Каждая стратегия отвечает за одну сторону объекта (behavior, scheduler, metrics, tuner, logging и т.д.).

- случайно увидел вот это.
Вы имеете в виду что и logging необходимо как-то отделять от всего остального ?
Я делал такое - передавал в другой специальный Backgroundservice высвечивать ЛОги в консоли в его собственном отелльном от всего остального контексте исполнения

Да, именно так. Я имел в виду, что logging — это отдельная сторона объекта, и её тоже стоит отделять от поведения, метрик, тунинга и т.д.

В вашем случае:

Я делал такое — передавал в другой специальный BackgroundService высвечивать логи в консоли в его собственном отдельном от всего остального контексте исполнения.

Это идеально отражает принцип:

- Объект/сервис — делает работу (behavior, scheduler, metrics и т.д.).
- `LoggingService` — занимается только логированием, в своём контексте, отдельно от логики бизнеса.

Почему это правильно

1. One responsibility per strategy / service

- Основной сервис: «что делать и как».
- `LoggingService`: «куда и как писать логи».
- `MetricsService`: «что собирать и как».
- И т.д.

2. Понятная архитектура

- Новая команда или разработчик сразу видит:
 - Вот бизнес-логика,
 - Вот логирование,
 - Вот метрики.
- Нет «смешанного» кода, где логирование зашито внутри стратегий поведения.

3. Заменяемость

- Можно легко заменить:
 - `ConsoleLogger` → `FileLogger`,
 - `FileLogger` → `RemoteLogger`,
 - и т.д.,
- Не меняя код бизнес-стратегий.

4. Тестирование

- Поведение тестируется без логики логирования.
- Логирование тестируется отдельно (например, проверяется, что в консоль пишется корректный текст).

5. Для AI-контекста

- Когда вы потом собираете данные с разных стратегий и передаёте в модель, у вас:
 - Чёткие разделы: behavior, scheduler, metrics, tuner, logging.
 - Модель видит структурированную историю, а не одну «кашу».
- Это напрямую влияет на качество предсказаний и интерпретируемость.

Как это выглядит в концепции стратегий

logging можно рассматривать как стратегию:

```
public interface ILoggingStrategy
{
    void Log(string message, LogLevel level);
    void LogException(Exception ex, string message);
}

public class ConsoleLoggingStrategy : ILoggingStrategy
{
    public void Log(string message, LogLevel level)
    {
        Console.WriteLine($"{level}: {message}");
    }

    public void LogException(Exception ex, string message)
    {
        Console.WriteLine($"{LogLevel.Error}: {message}");
        Console.WriteLine(ex.ToString());
    }
}

public class BackgroundServiceWithLogging<TBehavior, TScheduler, TMetrics, TTuner, TLogging>
    : BackgroundService
    where TBehavior : IBehaviorStrategy
    where TScheduler : ISchedulerStrategy
    where TMetrics : IMetricsStrategy
    where TTuner : ITunerStrategy
    where TLogging : ILoggingStrategy
{
    private readonly TLogging _logging;

    public BackgroundServiceWithLogging(
        TBehavior behavior,
        TScheduler scheduler,
        TMetrics metrics,
        TTuner tuner,
        TLogging logging)
    {
        _behavior = behavior;
        _scheduler = scheduler;
        _metrics = metrics;
        _tuner = tuner;
        _logging = logging;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        _logging.Log("Start", LogLevel.Info);
        try
        {
            await _scheduler.StartLoopAsync(stoppingToken, async token =>
            {
                try
            
```

```

        {
            await _behavior.ExecuteAsync(token);
            _logging.Log("Step completed", LogLevel.Info);
        }
        catch (Exception ex)
        {
            _logging.LogException(ex, "Error during step");
        }
    });
}
finally
{
    _logging.Log("Stop", LogLevel.Info);
}
}
}

```

А можно, как вы и делали, **отдельным BackgroundService**:

```

public class LoggingFrontendService : BackgroundService
{
    private readonly ILoggingStrategy _logging;
    private readonly Channel<LogEntry> _logs;

    public LoggingFrontendService(ILoggingStrategy logging)
    {
        _logging = logging;
        _logs = Channel.CreateUnbounded<LogEntry>();
    }

    public void Enqueue(LogEntry entry) => _logs.Writer.WriteAsync(entry);

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            var entry = await _logs.Reader.ReadAsync(stoppingToken);
            _logging.Log(entry.Message, entry.Level);
        }
    }
}

```

- Бизнес-сервисы пишут логи в канал.
- LoggingFrontendService — только печатает/транслирует их.

Итог

- logging — это **отдельная сторона**, её тоже стоит отделять.
- Ваш подход (отдельный BackgroundService для логов) — это правильное применение принципа:
 - одна ответственность — один сервис/стратегия.

- Это проще для понимания, тестирования и, в вашем случае, для AI-интеграции, где важна структура данных.

✱

Но , к сожалению потом я обнаружил, что современные системы Логирования сами начали делать такие трюки - Elastic Search итд

Да, современные системы логирования (Elastic Search, Logstash, Fluentd, Graylog и т.п.) действительно сами реализуют ту же идею, которую вы описали:

- Они отделяют:
 - сбор логов (*ingest*),
 - маршрутизацию и фильтрацию,
 - хранение,
 - отображение и поиск.
- И часто используют собственные пайплайны и стратегии внутри себя.

Но это не отменяет ваш подход, а наоборот:

1. Вы model-им архитектуру системы, а не копируете её

Ваша цель:

- Разделить объект по аспектам: behavior, scheduler, metrics, tuner, logging и т.д.
- Передать эти аспекты в AI-модель как структурированные признаки.

Системы вроде Elastic:

- Разделяют технические аспекты логинг-пайплайна: сбор, буферизация, фильтрация, хранение.
- Их «стратегии» — внутри системы, не для вашей бизнес-логики.

Вы не копируете Elastic, вы моделируете свою систему с разделением по аспектам.

2. Вы можете использовать Elastic как «внешнюю стратегию»

Ваш подход:

```
public interface ILoggingStrategy
{
    void Log(string message, LogLevel level);
}
```

Вы можете реализовать:

```

public class ElasticLoggingStrategy : ILoggingStrategy
{
    private readonly IElasticClient _client;

    public ElasticLoggingStrategy(IElasticClient client)
    {
        _client = client;
    }

    public void Log(string message, LogLevel level)
    {
        // отправляем в Elastic
    }
}

```

И тогда:

- Внутри вашей архитектуры: ****jednostna стратегия**** ILoggingStrategy.
- Внутри этой стратегии: **Elastic Search** как «инфраструктура».

То есть:

- Вы не смешиваете логику поведения и логику логирования.
- Вы не встраиваете Elastic прямо в бизнес-сервисы.
- Вы просто используете Elastic как реализацию вашей стратегии.

3. Ваш подход всё ещё правильный

Ваша идея:

- Разделить complex объект на независимые стратегии.
- Сбор всех их данных в единый контекст для AI.
- Не смешивать их внутри одного контейнера.

Это:

- Хорошо для архитектуры (SRP, композиция, тестируемость).
- Хорошо для AI (структурированные признаки, понятная семантика).
- Хорошо для будущего расширения (можно менять логи, метрики, scheduler без изменения поведения).

Elastic и подобные системы:

- Просто реализуют свою часть этой идеи на уровне инфраструктуры.
- Не заменяют вашу архитектуру, а поддерживают её.

4. Как это можно использовать фактически

Вы можете:

1. Оставить `ImageStrategy` (или `ILoggingStrategy`) как внутренний интерфейс.
2. Реализовать:
 - `ConsoleLoggingStrategy` (для тестов, локальной разработки),
 - `ElasticLoggingStrategy` (для продакшена).
3. В DI:
 - Локально: `AddSingleton<ILoggingStrategy, ConsoleLoggingStrategy>()`,
 - На проде: `AddSingleton<ILoggingStrategy, ElasticLoggingStrategy>()`.

И ваша архитектура:

- Не меняется от того, что используете Elastic.
- Остаётся чистой и структурированной.
- AI-модель продолжает получать данные в том же структурированном виде.

Итог

- Современные системы логирования действительно делают то, что вы описали — но на уровне **инфраструктуры**, а не вашей бизнес-логики.
- Ваш подход:
 - Разделять объект по аспектам,
 - Не смешивать их в одном контейнер,
 - Передавать в AI как структурированные данные,— остаётся **правильным** и даже **выигрышным**, независимо от того, что используете: консоль, файловый логер, Elastic или другую систему.

В связи с тем, что информации по коду и вообще программированию становится все больше и больше. Проекты стали большими - не большими монолитами, а большими в смысле множества в них микро - проектов.

Посоветуйте простую систему хранилища программистских Артефактов.

Я пока остановился на двух вариантах `notepad++` или `vscode`.

Для меня важны

- многоязычность
- возможность хранения кода

- возможность хранения описания проектов
- возможность сохранения snippets и все такое.

Сам я работаю на Visual studio.

Для кода - это нормально, а для описания - уже не то.

Еще смотрел от Microsoft - какой-то инструмент Что-то вроде one-notes - не подошло

Еще у VS Code - есть Noteboor

Что посоветуете ?

Ваша задача — не «сервер артефактов сборки» (как GitFlic Atlas, Ларец и т.п.), а **личная система хранения знаний и кода**: проектные описания, заметки, snippets, примеры на разных языках, схема архитектуры и т.п.

Для этого важнее:

- многоязычность кода и текста,
- удобное хранение,
- возможность иметь структуру по проектам,
- интеграция с тем, что вы уже используете (Visual Studio, VS Code).

Below — несколько простых вариантов, от «почти без настройки» до «чуть более продвинутой».

1. VS Code + Git (самый простой и практичный)

Если вы уже работаете в VS Code, это **идеальный вариант**.

Что получает:

- **Многоязычность:**
 - VS Code поддерживает почти все языки через расширения.
 - В одном workspace можно хранить проекты на C#, C++, JS, Python и т.д.
- **Хранение кода:**
 - У вас уже есть код в проектах.
 - Можно создать отдельный репозиторий code-snippets или dev-knowledge-base.
- **Хранение описаний проектов:**
 - В каждом проекте — README.md, docs/, TODO.md и т.п.
 - Markdown + VS Code отлично работают.
- **Snippets и заметки:**
 - В VS Code можно:
 - Создать проект Snippets с примерами по разным языкам.
 - В файлах .md хранить заметки, архитектурные схемы, решения.
 - Плюс есть встроенные user snippets (для быстрой генерации кода).

Как можно организовать:

```
dev-knowledge-base/  
  csharp/  
    BackgroundService/  
      README.md  
      samples/  
    DI/  
      README.md  
      samples/  
  cpp/  
    ...  
  js/  
    ...  
  notes/  
    architecture/  
      dapr/  
      blazor/  
    patterns/  
      strategy/  
      template-method/  
  snippets/  
    csharp/  
      background-service-sample.cs  
    js/  
      ...
```

Все это хранится в **Git-репозитории** (локально или на GitLab/GitHub).

Вы получаете:

- версиюность,
- поиск внутри проекта,
- возможность позже синхронизировать с облаком.

Почему VS Code, а не Notepad++

- Notepad++:
 - Хороший текстовый редактор, но не IDE.
 - Нет встроенной навигации по блокам, fold, расширений Jupyter/Notebook, удобной работы с Markdown.
- VS Code:
 - Есть поддержка Markdown, Git, расширений для разных языков.
 - Можно открывать несколько проектов в одном workspace.
 - Есть встроенный терминал, поиск, управление файлами.

2. VS Code + VS Code Notebooks (для «живых заметок с кодом»)

Вы уже посмотрели Notebooks — это хорошая идея, если вы хотите:

- иметь блокноты типа Jupyter, но с кодом разных языков.
- писать текст + примеры кода + выводы в одном файле (.ipynb).

В VS Code:

- Есть расширение Jupyter, которое позволяет:
 - открывать .ipynb как блокноты.
 - писать код на разных языках (C#, Python, JS) в ячейках.
 - сохранять результаты, графики, текстовые описания.

Как это можно использовать как хранилище артефактов:

- Создать репозиторий dev-notebooks:

```
dev-notebooks/  
  patterns/  
    strategy/  
      note.ipynb    # описание + примеры C# + выводы  
    template-method/  
      note.ipynb  
  architecture/  
    dapr/  
      dapr-intro.ipynb  
    blazor/  
      blazor-realtime.ipynb  
  snippets/  
    background-service/  
      background-service-pattern.ipynb
```

В ячейках:

- Текст: описание паттерна, где и как применяется, плюсы/минусы.
- Код: примеры на C#, C++, JS.
- Выводы: метрики, примеры, тесты.

Это удобно для изучения и повторения, особенно если вы потом будете использовать AI для анализа этих заметок.

3. VS Code + Markdown + Одно единое «Knowledge Base»

Если вы хотите что-то ближе к Wiki/Obsidian, но не OneNote:

- Создайте отдельный репозиторий dev-wiki или dev-knowledge.
- В каждом проекте — ссылки на другие заметки (через Markdown).
- Использовать VS Code как редактор и навигатор.

Пример:

```
dev-wiki/  
  patterns/  
    strategy/  
      index.md  
      пример-использования-в-backgroundservice.md  
    template-method/  
      index.md  
  architecture/  
    dapr/  
      intro.md  
      dapr-with-blazor.md  
    beras/  
      ...  
  ai/  
    как-использовать-ai-для-анализа-архитектуры.md  
    как-собрать-признаки-из-стратегий.md
```

В VS Code:

- Можно использовать **Markdown All in One**, **Markdown Preview Enhanced** и т.п.
- Есть быстрая навигация по заголовкам, Cabal, ссылки.

Это простой, но мощный вариант, особенно если:

- вы привыкли к Markdown,
- хотите иметь текстовую базу знаний, а не блокноты.

4. Если хочется что-то вроде Obsidian (но не VS Code Notebooks)

Если вы посмотрите Obsidian:

- Это локальное хранилище заметок в Markdown.
- В одном файле:
 - можно хранить текст,
 - код,
 - ссылки на другие заметки,
 - embed-ы изображений.
- Очень удобно:
 - для личной базы знаний,
 - для распределенных по проектам заметок.
- Есть:
 - граф связей между заметками,
 - поиск,

- плагины (например, для сниппетов, Diagrams и т.п.).

Вы можете:

- Хранить всё в Git (репозиторий `obsidian-kb`).
- И использовать VS Code как второй редактор для этого же набора файлов.

Но если вы уже привыкли к VS Code, Obsidian можно не вводить дополнительно, а просто использовать VS Code как «Obsidian-подобную» систему (Markdown + граф файлов).

5. Почему не OneNote / Microsoft-инструмент

OneNote и подобные:

- Хороши для общих заметок, но не для:
 - кода,
 - сниппетов,
 - версионности,
 - глубокой интеграции с репозиториями.
- Для программирования они не так удобны (код, форматирование, ссылки между проектами).

Если цель — программистская база знаний, лучше:

- Git + Markdown + VS Code,
- либо Git + Markdown + Obsidian.

6. Рекомендация для вашего случая

Учитывая:

- Вы работаете в Visual Studio.
- Уже смотрели VS Code и Notebooks.
- Нужно:
 - многоязычность,
 - хранение кода,
 - хранение описаний,
 - snippets.

Простая и эффективная система:

1. Основное хранилище — VS Code + Git.

- Создайте репозитории:
 - `dev-knowledge-base` (описания, паттерны, архитектура, AI-интеграции).

- `code-snippets` (примеры кода на разных языках).
- В каждом проекте:
 - `README.md` — описание.
 - `docs/` — детальные заметки.
 - `samples/` — примеры кода.

2. Для «живых заметок» с кодом:

- Используйте **VS Code Notebooks** (`.ipynb` файлы) для:
 - разбора паттернов,
 - архитектурных схем,
 - экспериментов с AI.

3. Для упрощения:

- Не вводить третьи инструменты (Obsidian, OneNote), а использовать VS Code как монолитную систему:
 - Текстовые заметки в Markdown.
 - Код в примерах.
 - Блокноты в Notebooks.

Это простая, гибкая и масштабируемая система, которая хорошо сочетается с вашим текущим стеком (Visual Studio + VS Code + GitLab).

✱

А что такое MArkdown ?

Markdown — это облегчённый язык разметки, который позволяет писать структурированный и красиво оформленный текст в обычных текстовых файлах (обычно с расширением `.md`), но без сложных инструментов вроде Word. [\[71\]](#) [\[72\]](#) [\[73\]](#)

Простыми словами:

- Вы пишете текст в простом редакторе.
- Используете специальные символы (например `#`, `*`, `-`, `**`), чтобы обозначить:
 - заголовки,
 - жирный/курсивный текст,
 - списки,
 - ссылки,
 - изображения,
 - блоки кода и т.д.
- А потом этот файл можно:
 - читать как обычный текст,

- конвертировать в HTML, PDF, Rich Text и другие форматы. ^[73] ^[71]

Почему это удобно программистам

1. Простой синтаксис

- Можно писать в любом текстовом редакторе (VS Code, Notepad++, Obsidian и т.п.).
- Файл `.md` открывается и понятен даже в виде исходного кода. ^[71]

2. Хорошо для документации

- В GitHub/GitLab почти все README `.md` — это Markdown.
- Описание проектов, паттернов, архитектурных решений удобно писать в `.md`. ^[74]

3. Поддержка кода

- Можно писать блоки кода с подсветкой:

```
```c#
public class Example
{
 public int Value { get; set; }
}
```

### 4. Гибкость

- Markdown легко конвертируется в HTML, PDF, Word и другие форматы с помощью инструментов (например, VS Code, pandoc, Obsidian). <sup>[71]</sup>

## Примеры синтаксиса

- Заголовки:

```
Заголовок 1 уровня
Заголовок 2 уровня
Заголовок 3 уровня
```

- Списки:

```
- Элемент 1
- Элемент 2
- Элемент 3
```

- Курсив и жирный:

```
курсив или _курсив_
жирный или __жирный__
```

- Ссылки:

[Название ссылки](https://example.com)

- Изображения:

- Блоки кода:

```
```c#  
var x = 10;
```

- Цитаты:

```
> Это цитата.  
> Ещё одна строка цитаты.
```

Где используется Markdown

- Документация в GitHub/GitLab (файлы README.md, CONTRIBUTING.md и т.п.).
- Блоги, форумы (например, Habr, Reddit, некоторые CMS).
- Инструменты для заметок: Obsidian, VS Code, Notion (в части), Markdown-редакторы.
- Архитектура и описания проектов (ваша задача — как раз это).

Для вашей задачи

Для хранения «программистских артефактов» (описаний, проектных заметок, паттернов, примеров кода):

- Вы можете создавать файлы README.md, notes.md, architecture.md в репозиториях.
- В VS Code:
 - Есть встроенная поддержка Markdown,
 - Можно открывать Preview (наглядно видеть оформленный текст),
 - Можно искать по файлам, ссылкам между заметками.

Это простая, универсальная и масштабируемая система, которая хорошо сочетается с Git и вашим текущим стеком (Visual Studio + VS Code).

✱

То есть Markdown - это упрощенный html

Не совсем так.

Markdown — это не упрощённый HTML, а упрощённый способ форматирования текста, который потом можно превратить в HTML.

Разница:

- **HTML:**

- Создаётся для *публикации* и *отображения* в браузере.

- Синтаксис тяжёлый, много символов (`<div>`, `<p>`, `<span>`, `<strong>` и т.д.)

- Исходный текст плохо читается без инструментов. ^[80] ^[81]

- **Markdown:**

- Создаётся для *написания* и *чтения* человеком.
- Синтаксис простой: #, *, -, ** и т.п.
- Исходный текст остаётся читаемым даже без преобразования. ^[81] ^[80]

Пример:

HTML:

```
<h1>Заголовок</h1>
<p>Это <strong>жирный</strong> текст.</p>
<ul>
  <li>Элемент 1</li>
  <li>Элемент 2</li>
</ul>
```

Markdown:

```
# Заголовок

Это **жирный** текст.

- Элемент 1
- Элемент 2
```

Markdown *можно* конвертировать в HTML (и в PDF, Rich Text и другие форматы), но:

- Его задача — не «сократить HTML»,
- а **удобно писать текст**, который потом система превратит в красивый документ или веб-страницу. ^[80] ^[81]

Так что правильнее сказать:

Markdown — это удобный способ разметки текста, который потом часто превращается в

HTML, но сам по себе HTML не является его «сутью».



Я использую heromix для хранения кода.

Какие еще варианты есть с такой функциональностью.

heromix - многословен, особенно в начале печатает большую шапку.

Это не очень удобно - все время эту шапку обирать.

Heromix — это инструмент, который упаковывает весь репозиторий в один файл (XML, Markdown, JSON, plain text), чтобы удобно передавать его в LLM (Claude, ChatGPT, Perplexity и т.п.). ^[88] ^[89]

Его основная ценность:

- «всё кода в одном файле» — удобно для AI-анализа,
- поддерживает многоязычность,
- можно настроить, что включать/исключать.

Но как вы заметили:

- в начале он печатает большую шапку (метаданные, описание, настройки),
- это неудобно, если вы постоянно хотите «обирать» шапку при копировании.

Альтернативы с похожей функциональностью

1. RepoPacker / Rerorack (форк/предшественник)

Heromix ранее назывался Rerorack (иногда такие форки существуют).

Функциональность очень похожа:

- Упаковка репозитория в один файл.
- Поддержка разных форматов.

Но:

- шапка тоже может быть большой, если не настроить.
- В целом, это почти тот же Heromix под другим именем.

Если вы хотите *тоже* упаковку, но с меньшим «шумом», проще:

- Настроить конфиг Heromix, чтобы шапка была минимальной,
- или использовать только нужные части (через `--include`, `--exclude`).

2. Написать свой простой упаковщик (самый гибкий вариант)

Для вас Repomix — это просто «собрать код в один файл».

Это можно реализовать очень просто:

Цель:

- собрать все *.cs, *.js, *.ts, *.md и т.д.
- в один Markdown-файл,
- с короткими заголовками:

```
# src/BackgroundService/MyService.cs

```c#
// код файла
```

Плюсы:

- Вы сами решаете, что писать в шапке.
- Можно сделать её одной строкой или совсем без неё.
- Можно добавить:
  - ссылки между файлами,
  - комментарии,
  - структуру проекта.

Пример простого подхода (концепт):

1. Скрипт (например, на C# или Python):

- проходит по директории,
- для каждого файла:
  - добавляет заголовок: # path/to/file.cs,
  - вставляет код в блок с подсветкой языка.

2. Вывод: hero-pack.md.

Это даст вам:

- тот же функционал, как Repomix,
- но ваш формат и без большой шапки.

### 3. Git + Markdown-база знаний (без единого файла)

Если проблема только в большой шапке, а не в том, чтобы «всё в одном файле», можно:

- Хранить код в **Git-репозиториях** (как у вас).
- Для AI-анализа:
  - передавать в модель отдельные файлы или группы файлов,
  - а не «весь реп в одном файле».

Это:

- избавит от большой шапки,
- даст гибкость:
  - можно собрать только нужные части (например, только `BackgroundService`, только `Patterns`),
  - без лишнего «мусора».

В современных LLM можно:

- загружать несколько файлов,
- или concatenate несколько файлов в один запрос, но своим выбором (без шапки).

### 4. Repo2Text / простые скрипты turia

Есть разные экспериментальные инструменты с названиями вроде:

- `repo2text`,
- `repo-to-onefile`,
- `git-archive-full`.

Они обычно:

- архивируют репозиторий в один текстовый файл,
- иногда с минимальной шапкой.

Но:

- они менее известны и стабильны, чем `Peromix`,
- часто требуют ручной настройки.

Если вы хотите попробовать:

- можно искать в GitHub по ключевым словам:
  - `repo to one file`,
  - `repository packer`,
  - `code to text`,
  - `repo2text`.

## 5. Использование VS Code + Git (без сторонних упаковщиков)

Учитывая ваш стек (Visual Studio + VS Code + GitLab):

- У вас уже есть:
  - код в репозиториях,
  - возможность использовать Git,
  - VS Code как редактор.

Можно:

- Создать отдельный репозиторий `code-snippets` или `dev-knowledge`.
- В нём хранить:
  - примеры кода (C#, C++, JS и т.д.),
  - Markdown-файлы с описаниями паттернов,
  - архитектурные решения.
- Для AI:
  - копировать в запрос выбранные файлы, а не «весь реп в одном файле».

Это:

- не требует больших шапок,
- даёт гибкость,
- работает с тем, что вы уже используете.

## Что можно сделать с Repomix прямо сейчас

Если вы хотите остаться на Repomix, но убрать/сжать шапку:

### 1. Настроить конфиг `repomix.config.json`:

- Уменьшить количество метаданных,
- Отключить не нужные части (например, статистику, длинные описания).

### 2. Использовать `--include` / `--exclude`:

- Упаковывать только нужные директории,
- чтобы шапка и контент были меньше.

### 3. Пост-обработка:

- Сделать простой скрипт, который:
  - убирает первую N строк (шапку),
  - или фильтрует по паттерну.



## Рекомендация

Учитывая:

- Вы уже используете Repomix для хранения кода и передачи в AI,
- Не хотите большую шапку,
- Работаете в VS Code + Visual Studio.

Варианты:

### 1. Оптимизировать Repomix:

- Через конфиг уменьшить шапку,
- Или использовать пост-скрипт для её удаления.

### 2. Сделать свой простой упаковщик:

- Очень простой скрипт (на C# или Python),
- который собирает код в Markdown-файл:
  - без большой шапки,
  - но с заголовками и подсветкой.

### 3. Перейти на Git + Markdown:

- Если не критично «всё в одном файле»,
- а только удобство для AI и хранения артефактов.

Если хотите, могу показать пример простого скрипта (C# или Python), который упаковывает код в Markdown без большой шапки.

✱✱

**" Если хотите, могу показать пример простого скрипта (C# или Python), который упаковывает код в Markdown без большой шапки."**

Покажите

Ниже два минималистичных варианта «своего Repomix» без шапки: на C# и на Python. Оба делают одно и то же:

- обходят директорию репозитория,
- выбирают файлы по маскам,
- собирают их в один .md файл,
- перед каждым файлом ставят заголовок и Markdown-блок кода.

Я специально сделал код максимально прямолинейным, чтобы легко адаптировать под ваши нужды.

## Вариант 1: C# (консольное приложение)

```
using System;
using System.IO;
using System.Linq;
using System.Collections.Generic;

class RepoToMarkdown
{
 static void Main(string[] args)
 {
 if (args.Length < 2)
 {
 Console.WriteLine("Usage: RepoToMarkdown <sourceDir> <outputFile.md>");
 return;
 }

 var sourceDir = args[0];
 var outputFile = args[1];

 if (!Directory.Exists(sourceDir))
 {
 Console.WriteLine($"Source directory does not exist: {sourceDir}");
 return;
 }

 // Какие расширения включать
 var extensions = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
 {
 ".cs", ".js", ".ts", ".cpp", ".h", ".json", ".yaml", ".yml", ".md"
 };

 using var writer = new StreamWriter(outputFile);

 // МИНИМАЛЬНАЯ шапка (можно вообще убрать)
 writer.WriteLine($"# Repository dump");
 writer.WriteLine();
 writer.WriteLine($"Source: {sourceDir}");
 writer.WriteLine($"Generated: {DateTime.Now}");
 writer.WriteLine();

 foreach (var file in Directory.EnumerateFiles(sourceDir, "**.*", SearchOption
 {
 var ext = Path.GetExtension(file);
 if (!extensions.Contains(ext))
 continue;

 var relativePath = Path.GetRelativePath(sourceDir, file);

 // Заголовок для файла
 writer.WriteLine();
 writer.WriteLine($"## {relativePath}");
 writer.WriteLine();

 // Определяем язык для Markdown-блока
 var lang = GetLanguageByExtension(ext);
```

```

 writer.WriteLine($"```${lang}");
 foreach (var line in File.ReadLines(file))
 {
 writer.WriteLine(line);
 }
 writer.WriteLine("```");
 writer.WriteLine();
 }

 Console.WriteLine($"Done: {outputFile}");
}

static string GetLanguageByExtension(string ext)
{
 return ext.ToLower() switch
 {
 ".cs" => "csharp",
 ".js" => "javascript",
 ".ts" => "typescript",
 ".cpp" => "cpp",
 ".h" => "cpp",
 ".json" => "json",
 ".yaml" => "yaml",
 ".yml" => "yaml",
 ".md" => "",
 _ => ""
 };
}
}

```

Как использовать:

1. Собрать как обычное консольное приложение .NET (например, `dotnet new console`).
2. Положить этот код в `Program.cs` (или отдельный файл и вызвать его).
3. Запустить:

```
dotnet run -- "C:\path\to\repo" "C:\path\to\dump.md"
```

Результат:

- `dump.md` без большой шапки от Repomix.
- Файлы идут один за другим:

```

src/Services/MyBackgroundService.cs

```csharp
// код файла
...

```

src/Controllers/HomeController.cs

```
// код файла
...
```

Шапку в начале (`# Repository dump` и пару строк) можете вообще убрать, если не нужны

```
***
```

```
## Вариант 2: Python (скрипт в одну файл)
```

Если вам удобнее Python (например, через VS Code или отдельную утилиту):

```
```python
import os
import sys
from datetime import datetime

EXTENSIONS = {
 ".cs": ".js", ".ts", ".cpp", ".h", ".json", ".yaml", ".yml", ".md"
}

LANG_MAP = {
 ".cs": "csharp",
 ".js": "javascript",
 ".ts": "typescript",
 ".cpp": "cpp",
 ".h": "cpp",
 ".json": "json",
 ".yaml": "yaml",
 ".yml": "yaml",
 ".md": ""
}

def get_lang(ext: str) -> str:
 return LANG_MAP.get(ext.lower(), "")

def repo_to_markdown(source_dir: str, output_file: str) -> None:
 if not os.path.isdir(source_dir):
 print(f"Source directory does not exist: {source_dir}")
 return

 with open(output_file, "w", encoding="utf-8") as out:
 # Минимальная шапка (можно удалить)
 out.write("# Repository dump\n\n")
 out.write(f"Source: {source_dir}\n")
 out.write(f"Generated: {datetime.now()}\n\n")

 for root, dirs, files in os.walk(source_dir):
 for name in files:
 ext = os.path.splitext(name)[1]
 if ext not in EXTENSIONS:
 continue
```

```

 full_path = os.path.join(root, name)
 rel_path = os.path.relpath(full_path, source_dir)

 out.write(f"\n## {rel_path}\n\n")

 lang = get_lang(ext)
 out.write(f"```{lang}\n")

 with open(full_path, "r", encoding="utf-8", errors="ignore") as f:
 for line in f:
 out.write(line)
 out.write("\n```\n\n")

 print(f"Done: {output_file}")

if __name__ == "__main__":
 if len(sys.argv) < 3:
 print("Usage: python repo_to_md.py <sourceDir> <outputFile.md>")
 sys.exit(1)

 source = sys.argv[1]
 output = sys.argv[2]
 repo_to_markdown(source, output)

```

Запуск:

```
python repo_to_md.py "C:\path\to\repo" "C:\path\to\dump.md"
```

## Что можно дальше улучшить под ваши задачи

Для вас, как .NET-разработчика, логично:

- расширить список расширений (.csproj, .sln, .razor, .cshtml, .config и т.д.),
- добавить фильтры:
  - исключить bin/, obj/, .git/, node\_modules/, dist/ и т.п.,
- добавить минимальные метаданные (например, список проектов, дерево папок).

Главное — вы:

- полностью контролируете формат,
- избавляетесь от большой шапки Repomix,
- остаётесь в привычном стеке (C# или Python + VS Code).

Если хотите, можем отдельно усложнить скрипт до почти полного аналога Repomix: с игнор-файлами, конфигом, несколькими форматами вывода (Markdown, plain text).

# // Регистрация фоновых сервисов (порядок не важен)

```
builder.Services.AddHostedService<PeriodicProcessor>();
builder.Services.AddHostedService<ContinuousListener>();
builder.Services.AddHostedService<StartupInitializer>();
builder.Services.AddHostedService<CronJob>();
```

Сейчас вошел в свои папки из VS Code

У меня в комментариях в начале абзаца, который (абзац) приведен выше Каждое слово в комментариях высвечивается в квадрате желтого цвета. То есть каждое слово комментария в клетке желтого цвета.

Как убрать эти клетки ? Слова посадили в Клетки - хорошо звучит, выглядит не очень.

Это не проблема кода, а функция VS Code — подсветка Unicode-символов (желтые квадраты показывают символы, которые могут выглядеть как другие, или относятся к небазовым ASCII).

В комментариях с русским текстом она часто «выделяет» каждый символ отдельным квадратом, выглядит как «слова в клетках» именно так, как вы описали. [\[98\]](#) [\[99\]](#) [\[100\]](#)

## Как убрать эти желтые клетки

### Вариант 1: через всплывающую подсказку (быстро)

1. Наведите курсор на любой символ в желтом квадрате.
2. Появится всплывающая подсказка с объяснением и опцией настройки.
3. Нажмите на неё и выберите что-то вроде:
  - Disable highlighting of non basic ASCII characters  
или
  - Adjust settings → отключить подсветку.

Это сразу запишет нужный параметр в ваши настройки.

### Вариант 2: через настройки вручную

1. В VS Code открыть настройки:
  - Ctrl + , (Windows/Linux) или Cmd + , (macOS),
  - или File → Preferences → Settings.
2. В поиске настроек написать: unicodeHighlight.
3. Найдите и установите в false следующие параметры:
  - Editor: Unicode Highlight: Non Basic ASCII  
(editor.unicodeHighlight.nonBasicASCII)

- Editor: Unicode Highlight: Ambiguous Characters  
(editor.unicodeHighlight.ambiguousCharacters)
- Editor: Unicode Highlight: Invisible Characters  
(editor.unicodeHighlight.invisibleCharacters)

Если хотите отключить только русские буквы — достаточно:

```
{
 "editor.unicodeHighlight.nonBasicASCII": false
}
```

### Вариант 3: в settings.json

Если вы редактируете settings.json напрямую:

#### 1. Откройте settings.json:

- Ctrl + Shift + P → Preferences: Open Settings (JSON).

#### 2. Добавьте:

```
{
 "editor.unicodeHighlight.nonBasicASCII": false,
 "editor.unicodeHighlight.ambiguousCharacters": false,
 "editor.unicodeHighlight.invisibleCharacters": false
}
```

После этого желтые квадраты вокруг букв в комментариях исчезнут.



## В комментариях желтые квадраты исчезли, но появились в Логах на словах на русском языке

Да, это всё то же — Unicode Highlight, но теперь в терминалке/в логах (в выводе приложения).

В терминале VS Code тоже применяется эта логика, но настройки терминала/кинша иногда «перебивают» или у них свои флаги.

### Как убрать желтые квадраты в логах (в терминале)

#### Вариант 1: через всплывающую подсказку (внутри терминала)

1. В окне терминала (в логах) наведите курсор на любую букву в желтом квадрате.
2. Появится всплывашка с объяснением и опцией:
  - Disable highlighting of non-basic ASCII charactersили

- Adjust settings → отключить.

3. Нажмите на неё.

Это часто меняет именно настройки терминала, а не редактора кода.

## Вариант 2: настройки терминала вручную

1. Ctrl + , → откроются настройки.
2. Поиск по: `unicodeHighlight`.
3. Проверьте и установите в `false` параметры, относящиеся к терминалу:
  - Terminal: Unicode Highlight: Non Basic ASCII  
(`terminal.integrated.unicodeHighlight.nonBasicASCII`)
  - Terminal: Unicode Highlight: Ambiguous Characters  
(`terminal.integrated.unicodeHighlight.ambiguousCharacters`)  
(если такие есть в вашей версии).

Если в списке настроек не видите именно `terminal.integrated.unicodeHighlight...`, то:

## Вариант 3: в `settings.json`

1. Ctrl + Shift + P → Preferences: Open Settings (JSON).
2. Добавьте:

```
{
 "editor.unicodeHighlight.nonBasicASCII": false,
 "editor.unicodeHighlight.ambiguousCharacters": false,
 "editor.unicodeHighlight.invisibleCharacters": false,

 "terminal.integrated.unicodeHighlight.nonBasicASCII": false,
 "terminal.integrated.unicodeHighlight.ambiguousCharacters": false,
 "terminal.integrated.unicodeHighlight.invisibleCharacters": false
}
```

(если `terminal.integrated.unicodeHighlight...` не поддерживается вашей версией — просто не будет ошибки, настройки редактора всё равно отключат подсветку.)

## Вариант 4: добавить русскую локаль как разрешенную (более мягкий способ)

Если хотите оставить подсветку, но для русского текста:

```
{
 "editor.unicodeHighlight.allowedLocales": {
 "ru": true
 },
 "terminal.integrated.unicodeHighlight.allowedLocales": {
 "ru": true
 }
}
```



Это отключает выделение кириллицы, но другие символы могут продолжать подсвечиваться.

Если после включения этих настроек в терминале квадраты остались — попробуйте:

- перезапустить VS Code,
- или попробовать режим Developer: Reload Window (через Ctrl + Shift + P).

После этого желтые квадраты в логах на русском языке должны исчезнуть.

✱✱

1. <https://bool.dev/blog/detail/pattern--shablonnyy-metod-template-method>
2. <https://radioprogram.ru/post/1506>
3. [https://jasulib.org/kg/wp-content/uploads/2024/02/Chistaya\\_arkhitektura\\_Iskusstvo\\_razrabotki\\_programmnogo\\_obespechenia.pdf](https://jasulib.org/kg/wp-content/uploads/2024/02/Chistaya_arkhitektura_Iskusstvo_razrabotki_programmnogo_obespechenia.pdf)
4. [https://library.tsilikin.ru/Техника/Программирование/Desing/Фундаментальный\\_подход\\_к\\_программной\\_архитектуре.pdf](https://library.tsilikin.ru/Техника/Программирование/Desing/Фундаментальный_подход_к_программной_архитектуре.pdf)
5. <https://dou.ua/forums/topic/27840/>
6. [https://disser.spbu.ru/files/2025/disser\\_kesiyana.pdf](https://disser.spbu.ru/files/2025/disser_kesiyana.pdf)
7. <https://javarush.com/all-articles>
8. [https://storage.tusur.ru/files/115518/2018\\_3.pdf](https://storage.tusur.ru/files/115518/2018_3.pdf)
9. <https://itproger.com/spravka/csharp/template-method>
10. <https://ru.stackoverflow.com/questions/352385/Метод-с-шаблонными-параметрами-реализовывающие-интерфейс>
11. <https://radioprogram.ru/post/1506>
12. <https://ru.stackoverflow.com/questions/1070936/В-каких-случаях-лучше-применять-template-method-а-в-каких-strategy>
13. <https://frontend-content.ru/patterns/behavioral/strategy/>
14. [https://logrocon.ru/news/design\\_patterns3](https://logrocon.ru/news/design_patterns3)
15. <https://ru.stackoverflow.com/questions/479412/с-Делегаты-плюсы-и-минусы>
16. <https://learn.microsoft.com/ru-ru/dotnet/csharp/programming-guide/delegates/>
17. <https://javarush.com/groups/posts/1035-raznica-mezhdu-patternami-sostojanie-i-strategija-v-java>
18. <https://ru.stackoverflow.com/questions/730936/Чем-отличается-паттерн-Фабричный-метод-от-паттерна-стратегии>
19. <https://sky.pro/wiki/java/razlichiya-mezhdu-patternami-factory-i-strategy-v-oop/>
20. [https://www.reddit.com/r/dotnet/comments/1cwdbpb/the\\_strategy\\_factory\\_pattern\\_combo/](https://www.reddit.com/r/dotnet/comments/1cwdbpb/the_strategy_factory_pattern_combo/)
21. <https://refactoring.guru/ru/design-patterns/strategy>
22. <https://radioprogram.ru/post/1504>
23. <https://zalki-lab.ru/php-pattern-strategy/>
24. [https://www.reddit.com/r/csharp/comments/1jx6kxj/strategy\\_pattern\\_vs\\_functional\\_objects/](https://www.reddit.com/r/csharp/comments/1jx6kxj/strategy_pattern_vs_functional_objects/)

25. [https://storage.piter.com/upload/contents/978549601649/978549601649\\_p.pdf](https://storage.piter.com/upload/contents/978549601649/978549601649_p.pdf)
26. <https://javarush.com/groups/posts/2271-pattern-proektirovanija-strategija>
27. <https://ru.stackoverflow.com/questions/593057/Является-ли-паттерн-проектирования-strategy-составной-частью-других-паттернов>
28. <https://bool.dev/blog/detail/pattern-strategiya-strategy>
29. <https://javarush.com/groups/posts/1035-raznica-mezhdu-patternami-sostojanie-i-strategija-v-java>
30. [https://logrocon.ru/news/design\\_patterns3](https://logrocon.ru/news/design_patterns3)
31. <https://learn.microsoft.com/ru-ru/dotnet/architecture/microservices/multi-container-microservice-net-applications/background-tasks-with-ihostedservice>
32. <https://oneuptime.com/blog/post/2025-12-23-set-up-background-services-aspnet-core/view>
33. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0>
34. <https://www.pvsm.ru/c-2/373643>
35. <https://ru.stackoverflow.com/questions/1282816/asp-net-core-ihosted-background-service-внезапно-выключается>
36. <https://medium.com/@vosarat1995/how-to-implement-a-net-background-service-for-continuous-work-in-2025-6929c496b62f>
37. <https://blog.jetbrains.com/dotnet/2023/05/09/dotnet-background-services/>
38. <https://habr.com/ru/articles/658847/comments/>
39. [https://www.youtube.com/watch?v=S0H\\_qElq59Y](https://www.youtube.com/watch?v=S0H_qElq59Y)
40. <https://github.com/dotnet/runtime/issues/43637>
41. <https://microarch.ru/blog/priznaki-chistoy-arkhitektury>
42. <https://refactoring.guru/ru/design-patterns/strategy>
43. <https://www.youtube.com/watch?v=13OfxIRBsO4>
44. [https://ru.wikipedia.org/wiki/Стратегия\\_\(шаблон\\_проектирования\)](https://ru.wikipedia.org/wiki/Стратегия_(шаблон_проектирования))
45. <https://ru.stackoverflow.com/questions/593057/Является-ли-паттерн-проектирования-strategy-составной-частью-других-паттернов>
46. <https://www.youtube.com/watch?v=1RxkK3iqx1A>
47. <https://bool.dev/blog/detail/pattern-strategiya-strategy>
48. [https://storage.piter.com/upload/contents/978549601649/978549601649\\_p.pdf](https://storage.piter.com/upload/contents/978549601649/978549601649_p.pdf)
49. <https://education.yandex.ru/handbook/flutter/article/clean-architecture>
50. <https://javarush.com/groups/posts/2271-pattern-proektirovanija-strategija>
51. <https://habr.com/ru/articles/135242/>
52. [http://grep.cs.msu.ru/python3.8\\_RU/digitology.tech/docs/python\\_3/howto/logging.html](http://grep.cs.msu.ru/python3.8_RU/digitology.tech/docs/python_3/howto/logging.html)
53. <https://www.datahata.by/info/articles/sistemyi-logirovaniya.html>
54. <https://habr.com/ru/companies/ultravds/articles/984372/>
55. <https://refactoring.guru/ru/design-patterns/strategy>
56. <https://www.youtube.com/watch?v=3ddPdk2NBtU>
57. <https://kubernetes.io/ru/docs/concepts/cluster-administration/logging/>
58. [http://grep.cs.msu.ru/python3.8\\_RU/digitology.tech/docs/python\\_3/howto/logging-cookbook.html](http://grep.cs.msu.ru/python3.8_RU/digitology.tech/docs/python_3/howto/logging-cookbook.html)

59. <https://www.mql5.com/ru/articles/18602>
60. <https://pythonru.com/primery/primery-logirovaniya-s-python-logging>
61. <https://www.computerra.ru/329044/kod-kontejnery-artefakty-kak-sobrat-vse-v-edinom-reestre/>
62. <https://python-school.ru/blog/news/vscode-vs-jupyter-notebook/>
63. <https://www.itc.by/luchshie-produkty-san-obespechivayushhie-realizaciyu-it-strategii-predpriyatiya/>
64. <https://diamant-storage.ru/flash-khranilishcha/>
65. <https://habr.com/ru/articles/841712/>
66. <https://k-integration.ru/product/hranilishhe-artefaktov-larecz/>
67. <https://visualstudio.microsoft.com/ru/vs/features/notebooks-at-microsoft/>
68. [https://marketing-tech.ru/it\\_company\\_product\\_type/sredstva-hraniya-dannyh/](https://marketing-tech.ru/it_company_product_type/sredstva-hraniya-dannyh/)
69. <https://ya.zerocoder.ru/open-notebook-cto-umeet-novaya-alternativa-notebook-lm/>
70. <https://www.okestorage.com/ru/автоматизированная-система-хранения>
71. <https://skillbox.ru/media/code/cto-takoe-markdown-i-zachem-on-nuzhen-redaktory-dialekty-otlichiya-ot-html/>
72. <https://lifehacker.ru/cto-takoe-markdown/>
73. <https://ru.wikipedia.org/wiki/Markdown>
74. <https://workspace.ru/blog/yazyk-razmetki-markdown-shpargalka-po-sintaksisu-s-primeram/>
75. <https://elma365.com/ru/help/platform/markdown.html>
76. <https://www.youtube.com/watch?v=phlbGdbYSJk>
77. <https://help.vivaldi.com/ru/services-ru/forum-ru/markdown-formatting/>
78. [https://www.reddit.com/r/ObsidianMD/comments/vkrjtd/what\\_is\\_markdown\\_and\\_where\\_can\\_i\\_learn\\_it/](https://www.reddit.com/r/ObsidianMD/comments/vkrjtd/what_is_markdown_and_where_can_i_learn_it/)
79. <https://www.mango-office.ru/journal/glossary/markdown/>
80. <https://skillbox.ru/media/code/cto-takoe-markdown-i-zachem-on-nuzhen-redaktory-dialekty-otlichiya-ot-html/>
81. <https://thecode.media/markdown/>
82. <https://ru.wikipedia.org/wiki/Markdown>
83. <https://lifehacker.ru/cto-takoe-markdown/>
84. <https://blog.skillfactory.ru/glossary/markdown/>
85. <https://ru.hexlet.io/blog/posts/cto-takoe-markdown-i-zachem-on-nuzhen>
86. [https://www.reddit.com/r/ObsidianMD/comments/vkrjtd/what\\_is\\_markdown\\_and\\_where\\_can\\_i\\_learn\\_it/](https://www.reddit.com/r/ObsidianMD/comments/vkrjtd/what_is_markdown_and_where_can_i_learn_it/)
87. <https://www.mango-office.ru/journal/glossary/markdown/>
88. <https://repomix.com/ru>
89. <https://repomix.com/ru/guide>
90. [https://habr.com/ru/companies/korus\\_consulting/articles/875992/](https://habr.com/ru/companies/korus_consulting/articles/875992/)
91. [https://www.reddit.com/r/brdev/comments/1jsf914/o\\_combo\\_mais\\_forte\\_atualmente\\_gemini\\_25\\_pro/](https://www.reddit.com/r/brdev/comments/1jsf914/o_combo_mais_forte_atualmente_gemini_25_pro/)
92. <https://github.com/yamadashy/repomix>
93. <https://comptercraft.ru/topic/2147-servisy-dlya-hraniya-fragmentov-koda/>
94. [https://gb.ru/posts/8\\_note\\_apps](https://gb.ru/posts/8_note_apps)
95. <https://aisharenet.com/ru/repomix/>

96. [https://www.reddit.com/r/learnprogramming/comments/zq8kio/notetaking\\_app\\_for\\_programmerstech\\_people/](https://www.reddit.com/r/learnprogramming/comments/zq8kio/notetaking_app_for_programmerstech_people/)
97. <https://repomix.com/ru/guide/use-cases>
98. [https://www.reddit.com/r/vscode/comments/rux509/some\\_characters\\_are\\_in\\_yellow\\_squares/](https://www.reddit.com/r/vscode/comments/rux509/some_characters_are_in_yellow_squares/)
99. <https://qna.habr.com/q/1089530>
100. <https://www.alucare.fr/ru/visual-studio-code-remove-square-желтый-акцент/>
101. <https://www.youtube.com/watch?v=zR9AwQMaQWE>
102. <https://otvet.mail.ru/question/234620590>
103. [https://skillbox.ru/media/base/goryachie\\_klavishi\\_v\\_vscode/](https://skillbox.ru/media/base/goryachie_klavishi_v_vscode/)
104. <https://ru.stackoverflow.com/questions/1365801/vs-code-Подсвечивание-кириллицы-в-html>
105. <https://www.youtube.com/watch?v=yTDiCe3PESk>
106. [https://www.reddit.com/r/vscode/comments/rux509/some\\_characters\\_are\\_in\\_yellow\\_squares/](https://www.reddit.com/r/vscode/comments/rux509/some_characters_are_in_yellow_squares/)
107. <https://otvet.mail.ru/question/238615758>
108. <https://www.alucare.fr/ru/visual-studio-code-remove-square-желтый-акцент/>
109. <https://skillbox.ru/media/code/visual-studio-code-ustanovka-nastroyka-rusifikatsiya-i-spisok-goryachikh-klavish/>
110. <https://gitverse.ru/blog/articles/development/241-visual-studio-code-ustanovka-nastrojka-rusifikaciya-i-rabota-v-vs-code>
111. <https://qna.habr.com/q/1089530>
112. <https://marketplace.visualstudio.com/items?itemName=MS-CEINTL.vscode-language-pack-ru>